

```
from google.colab import drive
drive.mount('/content/drive')

import zipfile, os
os.makedirs('/content/final_dataset', exist_ok=True)
with zipfile.ZipFile('/content/drive/MyDrive/final_dataset.zip', 'r') as z:
    z.extractall('/content/final_dataset')

FINAL_DIR = '/content/final_dataset'
```

Mounted at /content/drive

```
import tensorflow as tf
from tensorflow.keras import layers

IMG_SIZE, BATCH_SIZE, SEED = (224, 224), 32, 42

train_ds = tf.keras.utils.image_dataset_from_directory(
    FINAL_DIR, validation_split=0.2, subset="training", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE)
val_ds = tf.keras.utils.image_dataset_from_directory(
    FINAL_DIR, validation_split=0.2, subset="validation", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE)

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

Found 16767 files belonging to 4 classes.
Using 13414 files for training.
Found 16767 files belonging to 4 classes.
Using 3353 files for validation.

```
import tensorflow as tf
from tensorflow.keras import layers

class RandomGaussianBlur(layers.Layer):
    """Applies a real 3x3 Gaussian blur to ~prob fraction of batches during training."""
    def __init__(self, prob=0.3, **kwargs):
        super().__init__(**kwargs)
        self.prob = prob

    def call(self, images, training=None):
```

```

if not training:
    return images
apply_blur = tf.random.uniform([]) < self.prob
def blur():
    kernel = tf.constant([[1,2,1],[2,4,2],[1,2,1]], dtype=tf.float32) / 16.0
    kernel = tf.reshape(kernel, [3,3,1,1])
    kernel = tf.tile(kernel, [1,1,3,1])
    return tf.nn.depthwise_conv2d(images, kernel, strides=[1,1,1,1], padding='SAME')
return tf.cond(apply_blur, blur, lambda: images)

def get_config(self):
    config = super().get_config()
    config.update({"prob": self.prob})
    return config

```

```

model = tf.keras.models.load_model(
    '/content/drive/MyDrive/checkpoints/unified_baseline.keras',
    custom_objects={'RandomGaussianBlur': RandomGaussianBlur}
)

```

```
base_model = model.get_layer('mobilenetv2_1.00_224') # use actual printed name
```

```

base_model.trainable = True
for layer in base_model.layers[:100]:
    layer.trainable = False

```

```

model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])

```

```

callbacks = [
    tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),
    tf.keras.callbacks.ModelCheckpoint(
        '/content/drive/MyDrive/checkpoints/unified_finetune_latest.keras',
        save_best_only=False, save_freq='epoch'),
    tf.keras.callbacks.ReduceLROnPlateau(factor=0.2, patience=3)
]

```

```
history_fine = model.fit(train_ds, validation_data=val_ds, epochs=20, callbacks=callbacks)
```

Epoch 1/20

420/420  **2687s** 6s/step - accuracy: 0.8570 - loss: 0.4465 - val_accuracy: 0.9690 - val_loss: 0.0917 - learning_rate: 1.0000e-05

Epoch 2/20

420/420  **2703s** 6s/step - accuracy: 0.9333 - loss: 0.1769 - val_accuracy: 0.9758 - val_loss: 0.0634 - learning_rate: 1.0000e-05

Epoch 3/20

```

420/420 ————— 2689s 6s/step - accuracy: 0.9504 - loss: 0.1362 - val_accuracy: 0.9803 - val_loss: 0.0587 - learning_rate: 1.0000e-05
Epoch 4/20
420/420 ————— 2731s 6s/step - accuracy: 0.9524 - loss: 0.1244 - val_accuracy: 0.9773 - val_loss: 0.0620 - learning_rate: 1.0000e-05
Epoch 5/20
420/420 ————— 2635s 6s/step - accuracy: 0.9613 - loss: 0.1064 - val_accuracy: 0.9785 - val_loss: 0.0598 - learning_rate: 1.0000e-05
Epoch 6/20
420/420 ————— 2689s 6s/step - accuracy: 0.9641 - loss: 0.1001 - val_accuracy: 0.9815 - val_loss: 0.0522 - learning_rate: 1.0000e-05
Epoch 7/20
420/420 ————— 2638s 6s/step - accuracy: 0.9659 - loss: 0.0942 - val_accuracy: 0.9800 - val_loss: 0.0531 - learning_rate: 1.0000e-05
Epoch 8/20
420/420 ————— 2718s 6s/step - accuracy: 0.9712 - loss: 0.0807 - val_accuracy: 0.9794 - val_loss: 0.0552 - learning_rate: 1.0000e-05
Epoch 9/20
420/420 ————— 2668s 6s/step - accuracy: 0.9730 - loss: 0.0734 - val_accuracy: 0.9836 - val_loss: 0.0454 - learning_rate: 1.0000e-05
Epoch 10/20
420/420 ————— 2683s 6s/step - accuracy: 0.9764 - loss: 0.0659 - val_accuracy: 0.9839 - val_loss: 0.0456 - learning_rate: 1.0000e-05
Epoch 11/20
420/420 ————— 2642s 6s/step - accuracy: 0.9754 - loss: 0.0671 - val_accuracy: 0.9827 - val_loss: 0.0460 - learning_rate: 1.0000e-05
Epoch 12/20
420/420 ————— 2669s 6s/step - accuracy: 0.9782 - loss: 0.0619 - val_accuracy: 0.9821 - val_loss: 0.0543 - learning_rate: 1.0000e-05
Epoch 13/20
420/420 ————— 2665s 6s/step - accuracy: 0.9793 - loss: 0.0570 - val_accuracy: 0.9851 - val_loss: 0.0447 - learning_rate: 2.0000e-06
Epoch 14/20
420/420 ————— 2645s 6s/step - accuracy: 0.9788 - loss: 0.0583 - val_accuracy: 0.9860 - val_loss: 0.0423 - learning_rate: 2.0000e-06
Epoch 15/20
420/420 ————— 2667s 6s/step - accuracy: 0.9780 - loss: 0.0607 - val_accuracy: 0.9863 - val_loss: 0.0427 - learning_rate: 2.0000e-06
Epoch 16/20
307/420 ————— 10:07 5s/step - accuracy: 0.9793 - loss: 0.0626

```

```

# 1. Define the callbacks
callbacks = [
    tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),
    tf.keras.callbacks.ModelCheckpoint(
        '/content/drive/MyDrive/checkpoints/unified_finetune_latest.keras',
        save_best_only=False,
        save_freq='epoch'
    ),
    tf.keras.callbacks.ReduceLROnPlateau(factor=0.2, patience=3)
]

# 2. Load the checkpoint saved at the end of epoch 15
model = tf.keras.models.load_model(
    '/content/drive/MyDrive/checkpoints/unified_finetune_latest.keras',
    custom_objects={'RandomGaussianBlur': RandomGaussianBlur}
)

# 3. Re-compile with the reduced learning rate
model.compile(

```

```

optimizer=tf.keras.optimizers.Adam(2e-6),
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)

# 4. Resume training from epoch 15 (runs epochs 16 to 20)
history_resume = model.fit(
    train_ds,
    validation_data=val_ds,
    initial_epoch=15,
    epochs=20,
    callbacks=callbacks
)

```

```

Epoch 16/20
420/420 ————— 2446s 6s/step - accuracy: 0.9825 - loss: 0.0486 - val_accuracy: 0.9863 - val_loss: 0.0403 - learning_rate: 2.0000e-06
Epoch 17/20
420/420 ————— 2514s 6s/step - accuracy: 0.9783 - loss: 0.0577 - val_accuracy: 0.9860 - val_loss: 0.0405 - learning_rate: 2.0000e-06
Epoch 18/20
420/420 ————— 2473s 6s/step - accuracy: 0.9793 - loss: 0.0570 - val_accuracy: 0.9869 - val_loss: 0.0395 - learning_rate: 2.0000e-06
Epoch 19/20
420/420 ————— 2536s 6s/step - accuracy: 0.9795 - loss: 0.0545 - val_accuracy: 0.9872 - val_loss: 0.0389 - learning_rate: 2.0000e-06
Epoch 20/20
420/420 ————— 2463s 6s/step - accuracy: 0.9808 - loss: 0.0530 - val_accuracy: 0.9860 - val_loss: 0.0395 - learning_rate: 2.0000e-06

```

```

# Save the FINAL best model – model holds best epoch's weights (restore_best_weights=True)
model.save('/content/drive/MyDrive/checkpoints/unified_best_model.keras')

```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

